

1. Introduzione

L'obiettivo dell'assignment è progettare e implementare una libreria concorrente per l'analisi ricorsiva del filesystem, in grado di:

- visitare una directory e tutte le sue sotto-directory
- classificare i file in fasce di dimensione (bands)
- aggregare i risultati in un report finale

Il problema è particolarmente interessante dal punto di vista della concorrenza perché combina:

- **operazioni I/O-bound** (lettura directory, lettura metadati file)
- **ricorsione potenzialmente profonda**
- **parallelizzazione naturale** (ogni directory è indipendente)
- **aggregazione di risultati parziali**

Sono state sviluppate tre versioni con approcci che se pur tutti concorrenti, molto diversi fra loro, ognuno dei quali presenta vantaggi e svantaggi che verranno esaminati in seguito.

2. Analisi del problema dal punto di vista concorrente

La scansione del filesystem presenta tre caratteristiche fondamentali:

Il problema è fortemente I/O-bound => L'operazione dominante è `listFiles()` / `readDir()`, che richiede accesso al disco. Il tempo di CPU è minimo, la latenza del disco è il fattore limitante. Questo è ancora più evidente su macchine con hard disk meccanici, dove il parallelismo permette di mascherare parzialmente la latenza e ridurre sensibilmente il tempo totale di scansione

La ricorsione è naturalmente parallelizzabile => Ogni directory può essere scansionata indipendentemente dalle altre, questo oltre a ridurre la complessità generale permette di lanciare task in modo parallelo.

È necessario aggregare risultati parziali => Ogni directory produce un report locale che deve essere combinato con quelli delle sotto-directory. Nelle versioni Rx e Virtual Threads, questi report vengono generati da thread diversi, quindi è necessario che siano immutabili: nessun thread deve modificare lo stesso oggetto, altrimenti si introdurrebbero race condition. L'immutabilità garantisce che ogni merge sia sicuro e privo di interferenze. Nella versione Vert.x, invece, il report è mutabile, ma viene aggiornato esclusivamente dall'event-loop, che è single-thread, il che elimina alla radice ogni problema di concorrenza.

3. Strategie adottate

3.1 Versione Event-Loop (Vert.x)

La versione Vert.x è costruita attorno al modello event-driven, basato su:

- un event-loop (single thread)
- un pool di background thread (worker) per operazioni bloccanti
- callback e future non bloccanti

Struttura:

FSStatLib espone il metodo `getFSReport()` che avvia direttamente la pipeline asincrona sull'event-loop tramite `compose()` e `recover()`, restituendo subito una `Future`. `fs.readDir()` e `fs.props()` sono chiamate asincrone delegate ai worker thread; i risultati tornano sempre sull'event-loop tramite callback. L'event-loop aggiorna un `FSReport` mutabile, sicuro perché aggiornato da un solo thread.

Ricorsione:

La scansione ricorsiva è strutturata attorno a due metodi privati: `scanDirectory()` e `processEntry()`.

1. **Lettura asincrona della directory:** La chiamata a `fs.readDir(dir)` è completamente non bloccante: Vert.x delega l'operazione ai worker thread e l'event-loop continua a processare altri eventi. Il metodo `recover()` gestisce localmente le directory non accessibili, restituendo una lista vuota invece di propagare l'errore all'intera scansione
2. **Ogni entry produce una Future:** Tramite `processEntry()`, ogni entry che può essere o file o directory produce un'unica `Future`. Se l'entry è un file, viene creato un report con quel solo file e wrappato in `Future.succeededFuture()`. Se è una directory, viene avviata ricorsivamente `scanDirectory()`.
3. **Sincronizzazione finale:** `Future.all(futures)` attende il completamento parallelo di tutte le entry. Non è una chiamata bloccante: registra una callback che l'event-loop eseguirà quando tutte le future saranno in stato `succeeded`. A quel punto viene eseguito il merge bottom-up: ogni sotto-report viene fuso nel report della directory padre, che a sua volta diventerà input del merge del livello superiore. La composizione della pipeline avviene tramite `compose()` per concatenare operazioni asincrone e `recover()` per assorbire errori localmente e continuare la pipeline, e `Future.succeededFuture()` per wrappare un valore sincrono in una `Future` quando il risultato è già disponibile senza I/O.

Estensione interattiva:

L'estensione interattiva (`FSStatLibInteractive`) mantiene lo stesso modello a event-loop della versione base. Le differenze principali sono le seguenti:

- **Report live:** Un FSReport globale (liveReport) viene aggiornato ad ogni sotto-albero completato, nel map() di Future.all(). È mutabile ma acceduto solo dall'event-loop, quindi senza race condition. Un metodo getLiveReport() espone uno “snapshot” del report corrente.
- **Un Timer:** Invece di notificare la GUI ad ogni file trovato (il che provocherebbe migliaia di invokeLater sull'EDT) viene usato vertx.setPeriodic() per scattare uno snapshot del liveReport ogni 200ms tramite getLiveReport() e passarlo alla callback onUpdate. Il timer viene cancellato al completamento della scansione.
- **Stop cooperativo:** Un AtomicBoolean è necessario perché stop() viene chiamato dall'EDT mentre la scansione gira sull'event-loop: serve visibilità cross-thread senza sincronizzazione esplicita. Il flag viene controllato in più punti della pipeline in modo da interrompere sia il lancio di nuove operazioni I/O che il merge dei risultati già completati. Le operazioni I/O già “in volo” non possono essere cancellate.

3.2 Versione Reactive (RxJava)

La versione basata su **RxJava** affronta la scansione del filesystem adottando un modello completamente diverso rispetto al precedente. L'idea centrale è trasformare l'intero albero delle directory in un *Flowable<File>* che emette tutti i file presenti nella directory radice e nelle sue sotto-directory.

Pipeline reattiva:

La costruzione del flusso avviene tramite il metodo scanDir(), che utilizza Flowable.defer() per garantire che la scansione non inizi immediatamente, ma solo quando qualcuno si iscrive al flusso (flusso “cold”). Questo è un aspetto importante: la pipeline è definita in anticipo, ma l'esecuzione parte solo al momento della subscribe().

Per quanto riguarda la lettura delle directory viene sfruttato il metodo “.subscribeOn” sul quale viene specificato l'uso di uno scheduler specifico (Schedulers.io()). In questo modo ogni directory viene letta su uno scheduler I/O dedicato, permettendo di parallelizzare naturalmente la visita dell'albero. Tramite l'approccio RxJava ci possiamo permettere di non creare thread manualmente, la gestione della concorrenza viene delegata allo scheduler, che decide quando e come eseguire le operazioni, il tutto permette quindi di avere una visione più “ad alto livello” del problema.

Ricorsione tramite flatMap:

La ricorsione non è implementata tramite chiamate dirette, ma tramite la composizione dei flussi:

- se l'entry è un file => viene emesso come singolo elemento del flusso.
- se l'entry è una directory => scanDir() viene richiamato ricorsivamente e il flusso risultante viene “appiattito” nel flusso principale tramite flatMap.

Aggregazione funzionale tramite reduce:

Una volta ottenuto un flusso di file, la pipeline applica una trasformazione da file a file.length(), infine viene tutto aggregato in un unico report immutabile. Il metodo withFile() infatti crea un nuovo report immutabile ad ogni aggiornamento, poiché la pipeline può essere eseguita in parallelo, l'immutabilità garantisce che non ci siano race condition: ogni step produce un nuovo oggetto, e il reduce combina questi oggetti sfruttando uno stile funzionale.

Sottoscrizione e completamento:

La pipeline restituisce un Flowable<FSReportRx> che emette un solo elemento: il report finale. È importante sottolineare che il main utilizza blockingSubscribe() per attendere il completamento della pipeline, ma la libreria in sé rimane completamente non bloccante.

3.3 Versione Virtual Threads

La versione basata sui Virtual Threads sfrutta il modello di concorrenza che permette di creare un numero molto elevato di thread a costo estremamente ridotto. Questo consente di adottare un approccio “thread-per-task” anche in scenari ricorsivi e I/O-bound come la scansione del filesystem, mantenendo un codice imperativo semplice e leggibile, l'API per l'uso dei Virtual Threads rimane infatti molto simile a quella dell'uso di careers threads.

La struttura della libreria è centrata sul metodo getFSReport(), che crea un executor tramite Executors.newVirtualThreadPerTaskExecutor(). Questo executor non riutilizza i thread, bensì ogni chiamata a submit() crea un nuovo virtual thread dedicato. L'executor rimane attivo per tutta la durata della ricorsione e viene chiuso automaticamente al termine.

La logica principale risiede nel metodo ricorsivo scanDir(). Per ogni directory, il metodo:

1. **Legge le entry:** Se la directory non è accessibile, viene restituito un report vuoto.
2. **Processa i file localmente:** I file vengono gestiti immediatamente nel thread corrente, aggiornando il report immutabile tramite withFile(). Questo approccio mantiene il codice semplice e privo di effetti collaterali.
3. **Lancia la ricorsione parallela sulle sotto-directory:** Per ogni sotto-directory, viene creato un nuovo virtual thread tramite “executor.submit(() -> scanDir(entry, ...))”, la chiamata restituisce una Future<FSReportVT>, che rappresenta il risultato della scansione del sottoalbero, queste future vengono raccolte in una lista. Un aspetto importante da chiarire riguarda la differenza tra le Future utilizzate nella versione con Virtual Threads e quelle utilizzate nella versione Vert.x. Sebbene condividano il nome, si tratta di astrazioni profondamente diverse, progettate per modelli di concorrenza opposti. In questa versione vengono utilizzate le Future di java.util.concurrent, questa Future è **bloccante**: la chiamata a f.get() sospende il thread corrente finché il risultato non è disponibile. Nel contesto dei Virtual Threads questo non rappresenta un

problema, perché i Virtual Thread sono estremamente leggeri e possono essere sospesi e ripresi senza costi significativi. Nella versione con Vert.x vengono utilizzate le Future di io.vertx.core che è una Future completamente **non bloccante**, progettata per integrarsi con l'event-loop. Il risultato viene gestito tramite callback (onSuccess, onFailure) e propagato attraverso una catena di operazioni asincrone. L'event-loop non può mai essere bloccato, quindi l'intero modello si basa sulla composizione di callback e promise.

4. **Attende i risultati delle sotto-directory:** Dopo aver processato tutti i file locali, viene eseguita la fase di merge.

L'uso di un report immutabile (FSReportVT) è coerente con il modello: ogni virtual thread produce un proprio report locale, e il merge avviene solo dopo che i thread hanno terminato. Non vi è mai accesso concorrente allo stesso oggetto, e quindi non è necessaria alcuna forma di sincronizzazione. Nel complesso, la versione con Virtual Threads permette di esprimere la ricorsione in modo naturale, quasi identico alla versione sequenziale, ma con un parallelismo molto più efficace. Il codice rimane lineare e leggibile: non ci sono callback, non ci sono future non bloccanti, non ci sono scheduler o operatori reattivi. La concorrenza è ottenuta semplicemente lanciando un thread per ogni sotto-directory.

4. Conclusioni

L'analisi delle tre implementazioni mostra come modelli di concorrenza profondamente diversi possano affrontare lo stesso problema con approcci e risultati molto differenti.

La versione con **Vert.x**, basata su un event-loop e su operazioni asincrone delegate ai worker thread, evidenzia i punti di forza e i limiti del paradigma event-driven. Da un lato, il modello non bloccante permette di mantenere reattività e di integrare facilmente funzionalità interattive come aggiornamenti dinamici e stop cooperativo. Dall'altro, la necessità di comporre callback, e avere future non bloccanti introduce una complessità strutturale significativa che rende il codice più difficile da scrivere rispetto agli altri due approcci, non è semplice seguire il flusso del programma.

La versione con **RxJava** adotta un paradigma completamente diverso, modellando la scansione come un flusso di dati trasformato e aggregato tramite operatori funzionali. Questo approccio permette di esprimere la ricorsione e la parallelizzazione in modo dichiarativo, delegando la gestione dei thread allo scheduler I/O. La pipeline risulta compatta e modulare, e l'uso di un report immutabile elimina alla radice i problemi di concorrenza. Tuttavia, l'overhead introdotto dagli scheduler e la natura I/O-bound del problema rendono questa soluzione meno performante rispetto a un modello più diretto come quello dei Virtual Threads.

La versione basata sui **Virtual Threads** rappresenta l'approccio più naturale per questo tipo di problema. Il modello thread-per-task, reso possibile dalla leggerezza dei virtual thread,

permette di esprimere la ricorsione in modo quasi identico alla versione sequenziale, ma con un parallelismo molto più efficace. L'uso di future bloccanti non introduce penalizzazioni, poiché i virtual thread possono essere sospesi e ripresi senza costi significativi. Il risultato è un codice lineare, leggibile e privo di complessità strutturale, che sfrutta appieno il parallelismo disponibile senza richiedere callback, scheduler o strutture reattive.

In sintesi, le tre versioni mostrano come la scelta del modello concorrente debba essere guidata dalla natura del problema. Per scenari fortemente I/O-bound e ricorsivi, come la scansione del filesystem, i Virtual Threads offrono la soluzione più semplice ed efficiente. RxJava rappresenta un'alternativa elegante e funzionale, mentre Vert.x si distingue soprattutto per la capacità di integrare funzionalità interattive, pur non essendo il modello più adatto dal punto di vista prestazionale.